

Assignment VI: Advanced Control Flow in a Mini-RISC Processor

2-bit Branch Prediction, Speculative Execution, Interrupts and Exceptions

Department of Computer Science and Engineering

1. Background and Context

In previous assignments, you incrementally built a Mini-RISC processor with:

- Instruction execution and datapath control
- Pipelining with hazard detection and forwarding
- Cache-based memory hierarchy
- Branch prediction using static and dynamic predictors

At this stage, your processor executes instructions efficiently but still lacks **system-level control mechanisms** that exist in real processors.

This assignment strengthens your processor by adding:

- Robust branch prediction using a 2-bit saturating counter
- Speculative execution with precise recovery
- Interrupt handling
- Exception handling

This transforms your Mini-RISC CPU into a realistic, usable computing system.

2. Learning Objectives

After completing this assignment, you should be able to:

- Explain why 1-bit branch predictors fail for loops
- Implement a 2-bit saturating branch predictor
- Safely execute instructions speculatively
- Handle asynchronous interrupts and synchronous exceptions
- Preserve precise architectural state
- Analyze CPI impact due to mispredictions and interrupts

3. Control Hazards and Branch Prediction Recap

A control hazard occurs when the next PC depends on a branch outcome that is resolved late in the pipeline.

Without prediction:

- The pipeline must stall or flush
- Performance degrades significantly

With prediction:

- Instructions are fetched speculatively
- Incorrect speculation is repaired by flushing

Prediction affects **performance only**, never correctness.

4. 2-Bit Saturating Branch Predictor

4.1 Predictor States

Each branch instruction is associated with a 2-bit counter:

State	Meaning	Prediction
00	Strongly Not Taken	Not Taken
01	Weakly Not Taken	Not Taken
10	Weakly Taken	Taken
11	Strongly Taken	Taken

4.2 Prediction Rule

```
predict_taken = (counter >= 2)
```

4.3 Update Rule

```
if (actual_taken && counter < 3) counter++;
if (!actual_taken && counter > 0) counter--;
```

This predictor requires two consecutive wrong outcomes to flip prediction.

5. Worked Example: Loop Branch

Consider the branch outcome sequence:

T T T T T N

- First iteration: misprediction
- Predictor converges to strongly taken
- Loop exit: one misprediction

Total mispredictions = 2 (independent of loop length).

6. Speculative Execution and Recovery

- IF stage uses branch predictor to select PC
- EX stage resolves actual outcome
- On misprediction:
 - Pipeline is flushed
 - PC is corrected
 - Speculative instructions are discarded

Speculative instructions must not update architectural state.

7. Interrupts and Exceptions: Motivation

Real processors must handle:

- External events (timer, I/O) — **interrupts**
- Execution errors — **exceptions**

These events cause control flow changes outside the normal instruction stream.

8. Interrupts vs Exceptions

Feature	Interrupt	Exception
Source	External	Instruction
Timing	Asynchronous	Synchronous
Example	Timer	Divide by zero

9. Architectural Support Required

Add the following registers:

- EPC — Exception Program Counter
- CAUSE — Reason for interrupt/exception
- IE — Interrupt Enable flag

Add a new instruction:

IRET

10. Precise Interrupt Handling Sequence

When an interrupt or exception occurs:

1. Complete the current instruction
2. Save PC into EPC
3. Save reason into CAUSE
4. Disable interrupts
5. Flush pipeline
6. Jump to handler address

Execution resumes using IRET.

11. Interaction with Speculation

- Interrupts have higher priority than branch prediction
- All speculative instructions are discarded
- Predictor must not be updated using flushed instructions

This guarantees correctness.

12. What You Must Implement

1. 2-bit saturating branch predictor
2. Speculative fetch and recovery
3. Timer interrupt (every N cycles)
4. Exception handling:
 - Divide by zero
 - Illegal opcode
5. EPC, CAUSE, IE registers
6. IRET instruction

13. Performance and Correctness Metrics

Your simulator must print:

```
Total instructions
Total branches
Mispredictions
Branch penalty cycles
Total interrupts
Total exceptions
Interrupt latency
Total cycles
CPI
```

14. Mandatory Experiments

Run the following programs:

- Loop with predictable branch
- If-else with irregular branch

For each program:

- Compare 1-bit vs 2-bit predictors
- Enable and disable interrupts

15. Analysis Tasks

In your report, answer:

1. Why does the 2-bit predictor outperform the 1-bit predictor?
2. How does interrupt frequency affect CPI?
3. Why must interrupts be precise?
4. Why is speculation dangerous without flushing?

16. Complete C Reference Skeleton

```
1  /*
2      =====
3      Mini-RISC Processor      Assignment VI (Capstone)
4      -----
5      Features:
   - 2-stage pipeline (IF / EX)
```

```

6      - 2-bit saturating branch prediction
7      - Speculative execution + recovery
8      - Timer interrupt
9      - Exceptions (illegal opcode, divide-by-zero)
10     - Precise interrupts
11     =====
12     */
13
14 #include <stdio.h>
15 #include <stdlib.h>
16 #include <string.h>
17
18 /* ===== CONFIGURATION ===== */
19 #define MEM_SIZE      256
20 #define BP_SIZE       16
21 #define TIMER_PERIOD  20
22 #define INT_HANDLER   200
23
24 /* ===== OPCODES ===== */
25
26 #define OP_NOP  0x0
27 #define OP_LOAD 0x1
28 #define OP_STORE 0x2
29 #define OP_MOVI 0x3
30 #define OP_ADD  0x4
31 #define OP_SUB  0x5
32 #define OP_DIV  0x7
33 #define OP_CMP  0x9
34 #define OP_JMP  0xA
35 #define OP_JZ   0xB
36 #define OP_IRET 0xE
37 #define OP_HALT 0xF
38
39 /* ===== CPU STATE ===== */
40
41 typedef struct {
42     unsigned char R[4];
43     unsigned char ACC;
44     unsigned char PC;
45     unsigned char Z, C;
46
47     /* Interrupt state */
48     unsigned char EPC;
49     unsigned char CAUSE;
50     unsigned char IE;
51 } CPU;
52
53 CPU cpu;
54
55 /* ===== MEMORY ===== */

```

```

56
57 unsigned char IMEM[MEM_SIZE];
58
59 /* ===== PIPELINE ===== */
60
61 typedef struct {
62     unsigned char IR;
63     unsigned char PC;
64     int valid;
65     int predicted_taken;
66 } IFEX;
67
68 IFEX ifex;
69
70 /* ===== BRANCH PREDICTOR ===== */
71
72 typedef struct {
73     int valid;
74     int counter; /* 2-bit saturating counter: 0 3 */
75 } BPEnt;
76
77 BPEnt bp_table[BP_SIZE];
78
79 /* ===== PERFORMANCE COUNTERS ===== */
80
81 int cycles = 0;
82 int instructions = 0;
83
84 int total_branches = 0;
85 int mispredictions = 0;
86 int branch_penalty_cycles = 0;
87
88 int interrupts = 0;
89 int exceptions = 0;
90
91 /* ===== UTILITY ===== */
92
93 void reset_cpu() {
94     memset(&cpu, 0, sizeof(cpu));
95     memset(&ifex, 0, sizeof(ifex));
96     memset(bp_table, 0, sizeof(bp_table));
97     cpu.IE = 1;
98 }
99
100 void load_program(const char *file) {
101     FILE *fp = fopen(file, "r");
102     if (!fp) { printf("Program load error\n"); exit(1); }
103     int i = 0;
104     unsigned int x;
105     while (fscanf(fp, "%x", &x) != EOF)
106         IMEM[i++] = (unsigned char)x;

```

```
107     fclose(fp);
108 }
109
110 /* ===== BRANCH PREDICTION ===== */
111
112 int bp_index(unsigned char pc) {
113     return pc % BP_SIZE;
114 }
115
116 int predict_branch(unsigned char pc) {
117     int idx = bp_index(pc);
118     if (!bp_table[idx].valid)
119         return 0; /* default NT */
120     return (bp_table[idx].counter >= 2);
121 }
122
123 void update_predictor(unsigned char pc, int taken) {
124     int idx = bp_index(pc);
125     bp_table[idx].valid = 1;
126
127     if (taken && bp_table[idx].counter < 3)
128         bp_table[idx].counter++;
129     else if (!taken && bp_table[idx].counter > 0)
130         bp_table[idx].counter--;
131 }
132
133 /* ===== INTERRUPTS ===== */
134
135 int interrupt_pending() {
136     return (cpu.IE && (cycles % TIMER_PERIOD == 0));
137 }
138
139 void raise_interrupt(int cause) {
140     cpu.EPC = cpu.PC;
141     cpu.CAUSE = cause;
142     cpu.IE = 0;
143
144     ifex.valid = 0; /* flush pipeline */
145     cpu.PC = INT_HANDLER;
146     interrupts++;
147 }
148
149 /* ===== EXCEPTIONS ===== */
150
151 void raise_exception(int cause) {
152     cpu.EPC = cpu.PC;
153     cpu.CAUSE = cause;
154     cpu.IE = 0;
155
156     ifex.valid = 0;
157     cpu.PC = INT_HANDLER;
```



```
158     exceptions++;
159 }
160
161 /* ===== FETCH STAGE ===== */
162
163 void fetch_stage() {
164     ifex.IR = IMEM[cpu.PC];
165     ifex.PC = cpu.PC;
166     ifex.valid = 1;
167
168     unsigned char op = (ifex.IR >> 4) & 0xF;
169
170     if (op == OP_JMP || op == OP_JZ) {
171         int pred = predict_branch(cpu.PC);
172         ifex.predicted_taken = pred;
173         cpu.PC = pred ? (ifex.IR & 0xF) : cpu.PC + 1;
174     } else {
175         ifex.predicted_taken = 0;
176         cpu.PC++;
177     }
178 }
179
180 /* ===== EXECUTE STAGE ===== */
181
182 void execute_stage() {
183     if (!ifex.valid) return;
184
185     unsigned char ir = ifex.IR;
186     unsigned char op = (ir >> 4) & 0xF;
187     unsigned char arg = ir & 0xF;
188
189     instructions++;
190
191     switch (op) {
192
193     case OP_MOVI:
194         cpu.ACC = arg;
195         break;
196
197     case OP_ADD:
198         cpu.ACC += cpu.R[arg];
199         break;
200
201     case OP_SUB:
202         cpu.ACC -= cpu.R[arg];
203         break;
204
205     case OP_DIV:
206         if (cpu.R[arg] == 0) {
207             raise_exception(1); /* divide by zero */
208             return;
209         }
```

```

209     }
210     cpu.ACC /= cpu.R[arg];
211     break;
212
213 case OP_CMP:
214     cpu.Z = (cpu.ACC == cpu.R[arg]);
215     break;
216
217 case OP_JMP:
218 case OP_JZ: {
219     total_branches++;
220     int actual = (op == OP_JMP) ? 1 : cpu.Z;
221     int predicted = ifex.predicted_taken;
222
223     if (actual != predicted) {
224         mispredictions++;
225         branch_penalty_cycles += 2;
226         cpu.PC = actual ? arg : (ifex.PC + 1);
227         ifex.valid = 0;
228     }
229     update_predictor(ifex.PC, actual);
230     break;
231 }
232
233 case OP_IRET:
234     cpu.PC = cpu.EPC;
235     cpu.IE = 1;
236     break;
237
238 case OP_HALT:
239     printf("\n====_FINAL_REPORT_====\n");
240     printf("Instructions_:_%d\n", instructions);
241     printf("Branches_:_%d\n", total_branches);
242     printf("Mispredictions_:_%d\n", mispredictions);
243     printf("Branch_penalty_cyc.:_%d\n",
244            branch_penalty_cycles);
245     printf("Interrupts_handled_:_%d\n", interrupts);
246     printf("Exceptions_:_%d\n", exceptions);
247     printf("Total_cycles_:_%d\n", cycles);
248     printf("CPI_:_%f\n",
249            (float)cycles / instructions);
250     exit(0);
251
252 default:
253     raise_exception(2); /* illegal opcode */
254     return;
255 }
256
257 ifex.valid = 0;
258 }

```

```
259 /* ===== MAIN LOOP ===== */
260
261 int main() {
262
263     reset_cpu();
264     load_program("program.hex");
265
266     while (1) {
267         cycles++;
268
269         if (interrupt_pending()) {
270             raise_interrupt(0); /* timer */
271             continue;
272         }
273
274         execute_stage();
275         fetch_stage();
276     }
277     return 0;
278 }
```

listing

Sample Programs and Expected Outputs

To validate your implementation, you must test your processor using the following sample programs. These programs are designed to exercise:

- 2-bit branch prediction
- Speculative execution and recovery
- Timer interrupts
- Exceptions

You are **not expected** to match exact cycle counts, but the qualitative behavior must match.

Sample Program 1: Loop Branch with Timer Interrupt

Purpose: This program executes a loop with a highly predictable branch. A timer interrupt occurs periodically during loop execution.

Program

```
MOVI 5      ; ACC = 5
STORE R0    ; R0 = 5
```

```
LOOP:
LOAD R0
```

```
SUB R1      ; decrement
STORE R0
CMP R1
JZ LOOP     ; loop until zero
HALT
```

Expected Branch Behavior

- Branch is taken repeatedly, then not taken once
- 2-bit predictor converges to *strongly taken*
- Only two mispredictions should occur:
 - At loop entry
 - At loop exit

Expected Interrupt Behavior

- Timer interrupt triggers every N cycles
- Pipeline is flushed on interrupt
- PC is saved in EPC
- Interrupt handler executes
- IRET restores PC and resumes loop correctly

Expected Output (Qualitative)

```
Instructions      : > 20
Branches          : > 5
Mispredictions    : approximately 2
Interrupts handled : >= 1
Exceptions        : 0
CPI               : lower than 1-bit predictor
```

Sample Program 2: Divide-by-Zero Exception

Purpose: This program intentionally triggers a divide-by-zero exception.

Program

```
MOVI 10
STORE R0
```

```
MOVI 0
STORE R1
```

```
LOAD R0
```

```
DIV R1      ; divide by zero exception
ADD R2      ; must NOT execute
HALT
```

Expected Exception Behavior

- Divide-by-zero exception is detected in EX stage
- EPC stores the faulting instruction PC
- Pipeline is flushed
- Control jumps to exception handler
- ADD instruction after DIV is never committed

Expected Output (Qualitative)

```
Instructions      : < normal execution
Branches          : 0
Interrupts handled : 0
Exceptions        : 1
Program terminates via handler or HALT
```

17. Learning Outcome

After this assignment, you will have implemented the core control mechanisms used in modern RISC processors, including speculation, prediction, and asynchronous event handling.